

Building a MIPS Processor*

(*Well, a slightly simplified version that only supports the instructions lw, sw, beq, add, sub, and, or, and slt instructions.)

Introduction

In this lab you will do three things:

1. Define the control logic for the datapath control signals
2. Hookup the key parts of a MIPS processor to make a working datapath.
3. Verify that your design works by running a simple program

What to turn in

1. You are expected to turn in your completed mips-datapath-lab.circ file. You should rename it as: “**lastname-firstname-mips-datapath-lab.circ**” before you turn it in to simplify grading.
2. You are also required to turn in a pdf file in which you answer the Todos found within this document (name is as “**lastname-firstname-mips-datapath-lab.pdf**”).

Getting Started

Read the background material

Before you get started, we strongly recommend you read chapter 4.3 and 4.4 in the 5th edition of the book (Patterson & Hennessy, [Computer Organization and Design: The Hardware/Software Interface](#)). This will both show you pictures of what you are to build and explain how it works.

Files and programs

You are provided with the following files in the mips-datapath-lab.zip archive:

- mips-datapath-lab.circ: an eviscerated shell that is missing all the important parts of a MIPS processor
- 1234.mem: a hex memory file that contains the values 1, 2, 3, 4 in the first 4 words of memory
- test-code.mem: a file that contains a simple test program for verifying your design

To use these files you will need to install Logisim on your computer. We trust that you are capable of using Google to figure this out. Note that you may have to install a Java runtime to run Logisim. This is quite safe as long as you do not enable Java in your web browser.

Using Logisim

We recommend you watch the [Logisim](#) tutorials online before starting this lab (check [this one](#) for example). The key things for you to know before starting are:

1. How to wire up components in Logisim (drag with the arrow cursor) and how to add logic gates (click on them under “Gates” on the left).
2. How to edit the insides of components in logisim (double-click) and go back to the top view (click on “mips” on the left).
3. How to simulate in Logisim (choose “Simulation Enabled” and use the pointer cursor to change values).
4. How the clock works in Logisim (choose “Tick Once” to execute one clock cycle).

Open the `mips-datapath-lab.circ` file. You'll see something that looks a lot like the figure in the book:

This diagram illustrates the internal components and data flow of the MIPS processor. Key elements include:

- PC (Program Counter):** Provides the address for instruction memory.
- Instruction Memory:** Outputs instruction fields [31:0], [25:21], [20:16], [15:11], and [5:0] to various control and register units.
- Control Unit:** Receives instruction fields and outputs control signals: RegDst, Branch, MemRead, MemtoReg, ALUOp, MemWrite, ALUSrc, and RegWrite.
- Registers:**
 - Read register 1/2:** Output Read data 1/2 to the ALU.
 - Write register:** Receives data from the ALU and data memory to be stored.
 - Write data:** Receives data from the ALU and data memory to be stored.
- ALU (Arithmetic Logic Unit):**
 - ALU control:** Receives ALUOp and ALUSrc signals.
 - Shift left 2:** Shifts the Branch bit left by 2 positions.
 - Zero:** Outputs a Zero flag based on the ALU result.
 - ALU result:** The output of the ALU operation.
- Multiplexers (Mux):**
 - Mux 0:** Selects between the PC and the ALU result to update the PC.
 - Mux 1:** Selects between the ALU result and data from memory to be written to registers.
 - Mux 2:** Selects between the ALU result and data from memory to be written to the data memory.
- Adders:**
 - Add (4):** Adds a constant 4 to the PC.
 - Add (ALU result):** Adds the ALU result to the PC.
- Sign-extend:** Extends the 5-bit instruction field [5:0] to 32 bits.

Okay, so maybe they don't look all that similar, but you should be able to identify the same overall shape (the big loop on the top is part of the next PC logic) and the key components: PC, instruction memory, control logic, registers, ALU control, ALU, and data memory.

You probably noticed that the only wires we've included for you are the clock (at the bottom), a simplified next PC logic path (at the top), resets for the PC and register files (the red buttons), and one hint for getting started: we hooked up the RegWrite output from the Control logic to the RegWrite input on the Register file. (Wasn't that nice of us?) There are also several MUXes missing. Completing the datapath connections is the second part of this lab.

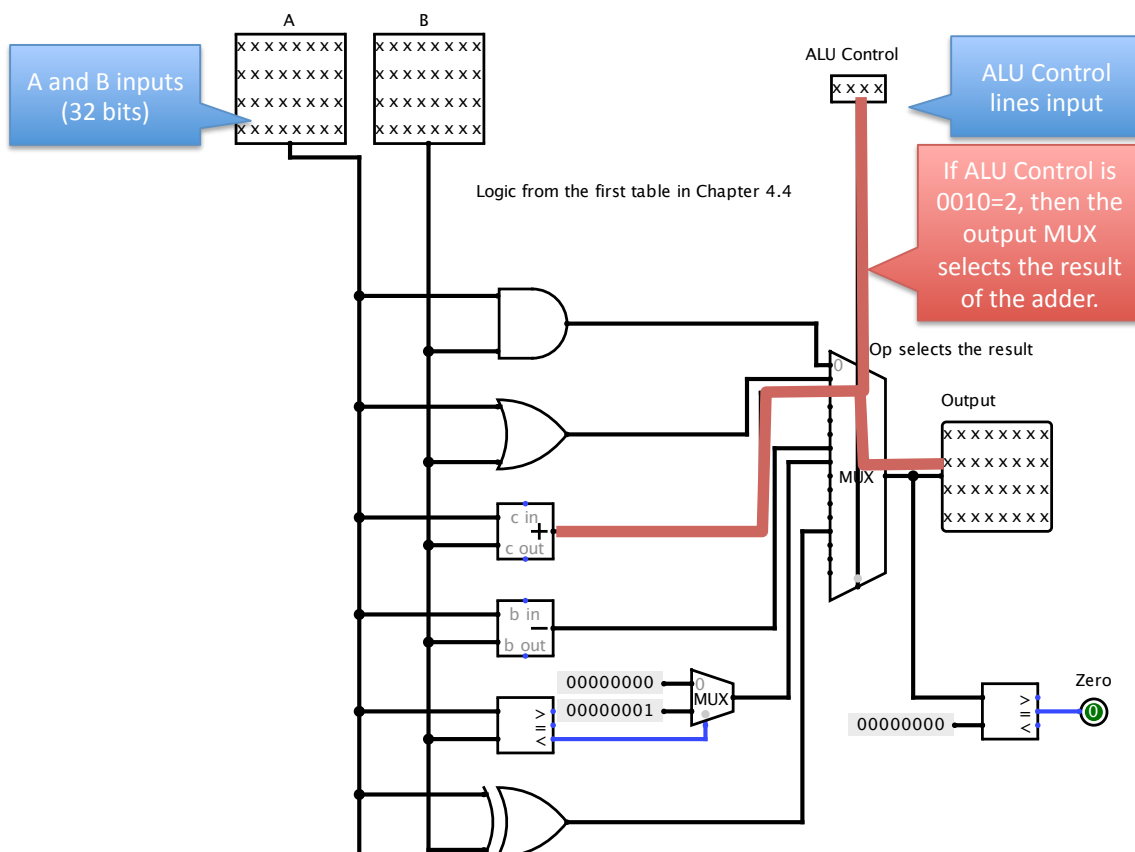
The first part of the lab is completing the control logic. That's right: those lovely ovals labeled "Control" and "ALU Control" are not finished on the inside. You will need to figure out the logic that goes inside of them.

A closer look

Before we get started on the actual work for the lab, let's take a slightly closer look at what you have. Right-click on the ALU and choose "View alu" or double-click on it in the hierarchy list on the left.

The ALU

Here you can see the ALU. It has two inputs (A and B) and calculates 6 functions:

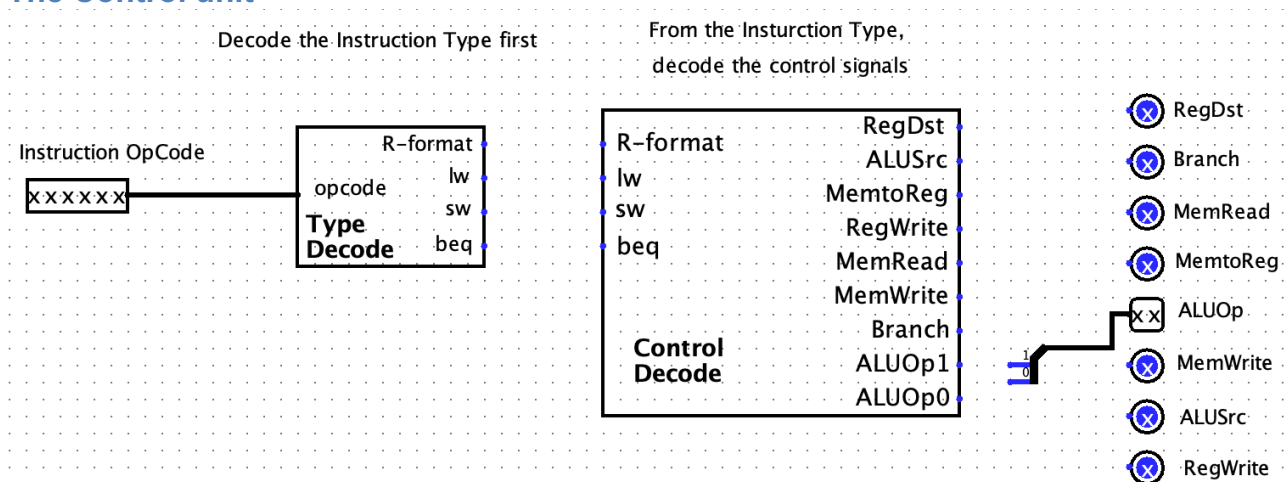


All 6 functions are calculated all the time, but the MUX at the end selects the right one based on the ALU control inputs. You can see from the figure that if the input to ALU control is 0010=2, then it will select input 2 (the third one since it starts counting at 0) and the output will be $A+B$. Compare this to the ALU function table below and make sure it makes sense to you. In particular, make sure you understand how the "set on less than" is implemented, since you will use that kind of logic later in the lab.

ALU control	Function
0000	AND
0001	OR
0010	Add
0110	Subtract
0111	Set on less than
1100	NOR

Now go back to the main view (double click on “mips” on the left) and then take a closer look at the “Control” unit.

The Control unit



You can see that the control unit has 4 parts:

1. The input: the instruction OpCode
2. The “Type Decode” which takes in the OpCode and outputs whether the instruction is an R-format, lw, sw, or beq instruction.
3. The “Control Decode” which takes in the type of instruction and outputs the correct control signals for the datapath.

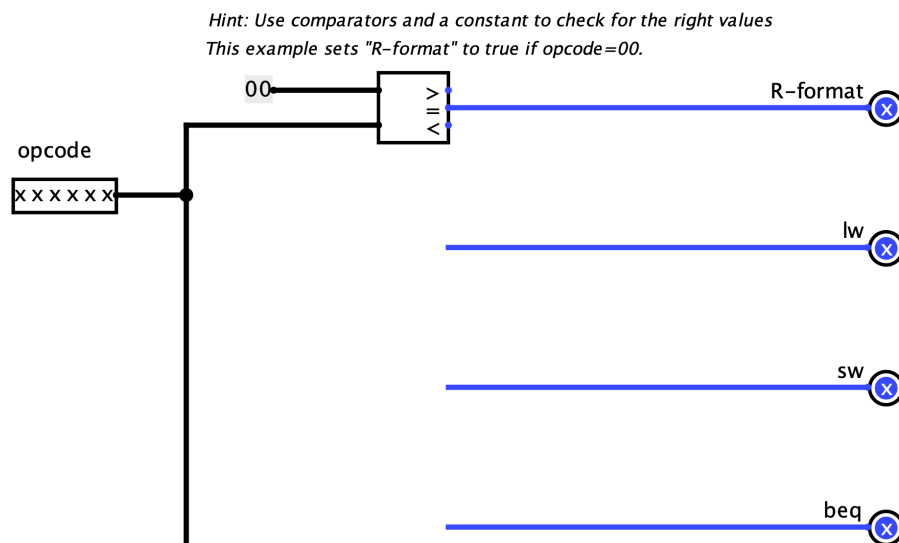
The Control unit’s job is to decode the instruction OpCode to control all the parts of the datapath. To do this we break up the decoding into two parts: first we decode the instruction type, then we use the instruction type to decode the control signals. This simplifies each piece of logic, which is great since you have to build them. (In practice, a CAD package will decide exactly how to break up the logic to make it as fast as possible.)

Part 1: Finishing the Control Logic

Now we’ll get down to the real work of making the control logic work. There are three parts you will need to complete: the **Type Decode unit**, the **Control Decode unit**, and the **ALU Control unit**. Each one uses a slightly different way to generate its output, so stay awake!

Start by taking a look inside the “Type Decode” unit:

The Instruction Type Decode Unit



Here you see that this unit is not complete. You will need to complete it to finish the lab. However, there's a hint about how to make this work. Notice how a comparator and a constant value are used at the top to determine if the instruction is R-format. What that logic does is determine if the OpCode equals 00. If it does, then the = output of the comparator will be true, and the R-format output of the block will be set to 1. Your job is to fill in the rest of this unit so it correctly asserts lw, sw, and beq when those types of instructions are encountered.

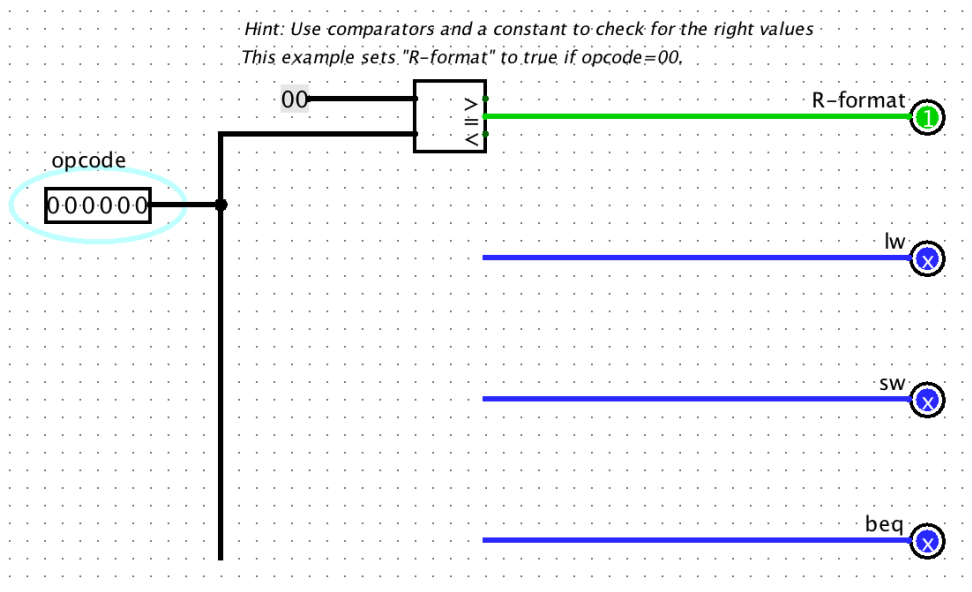
ToDo #1: Complete the Type Decoder Unit by adding comparators to detect the right instruction types. Take a screenshot of the completed unit and add it to the report.

To determine the instruction type you need the following information:

Input (opcode)	R-format	Lw	Sw	Beq
Op5	0	1	1	0
Op4	0	0	0	0
Op3	0	0	1	0
Op2	0	0	0	1
Op1	0	1	1	0
Op0	0	1	1	0

Testing

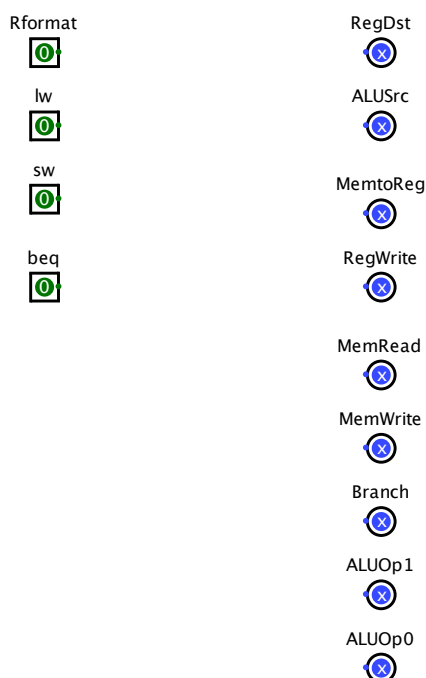
Once you have built the circuit you need to test it. To do this choose "Simulation Enabled" from the Simulate menu and use the hand pointer to set the inputs and make sure the right output is activated. Below I clicked on all the OpCode bits to make them 0 (don't worry about warnings that the state is set from elsewhere; that's just saying this block is hooked up). When they were all 0, the R-format output became 1 (light green) as we expect. Test all four inputs.



Testing the Type Decode Unit: I've put all zeros in the opcode input and it correctly enables only the R-format output.

The Control Decode Unit

Once you've completed the Instruction Type Decode unit you've got half of the Control logic finished. The other half is using the instruction type to determine the control signals to send. Take a look inside the control-decode unit to see what you've got to fill in there.



Well, this is what you'd expect: the inputs are the instruction type (on the left) and the control signals are the outputs (on the right). Now to generate this logic you can either stare really hard at the table below and figure it out yourself (which is fine) or you can use the built in "Combinatorial Analysis" tool in Logisim to build it for you.

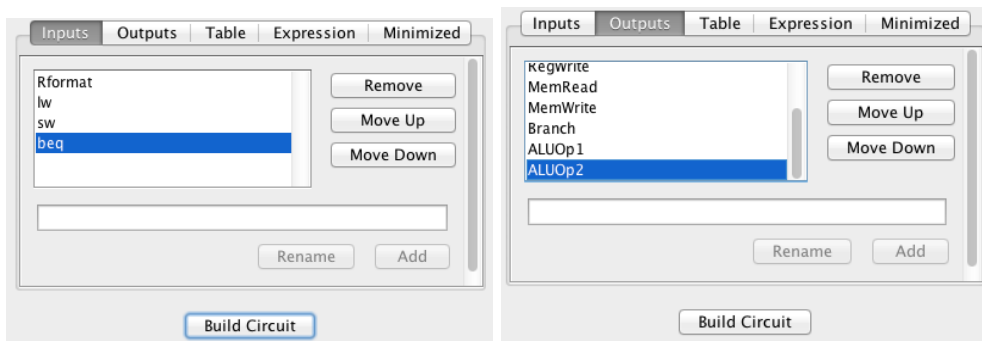
To do either, you need to know what the values should be. So, fill in the truth table below for each instruction type. Remember that you can (and should) put in an X for options that you don't care about. To do this you should think about what each instruction does. If you just copy this from the book or online you will have a much harder time debugging later because you won't understand as well.

Output	R-format	Lw	Sw	Beq
RegDst	1	0	X	X
ALUSrc				
MemtoReg				
RegWrite				
MemRead				
MemWrite				
Branch				
ALUOp1	1	0	0	0
ALUOp0	0	0	0	1

ToDo #2: Complete the above table to specify what the Control Decode Unit should do. Add it to the report.

Now that you've got the definition for the Control Decode Unit above, it's time to build it. Either just stare at it and figure out the logic (it's not that complicated) or use the Combinational Analysis tool under the Window menu.

For the Combinational Analysis Tool you first need to define the Inputs and Outputs of your circuit. These are the row and column headers from your truth table above:



Then enter the values into the Table section. Remember that if you don't care about a particular output you should just mark it as X. Click on the values to change them.

Inputs

Outputs

Table

Expression

Minimized

Rformat	lw	sw	beq	RegDst	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp1	ALUOp2
0	0	0	0									
0	0	0	1									
0	0	1	0									
0	0	1	1									
0	1	0	0									
0	1	0	1									
0	1	1	0									
0	1	1	1									
1	0	0	0									
1	0	0	1									
1	0	1	0									
1	0	1	1									
1	1	0	0									
1	1	0	1									
1	1	1	0									
1	1	1	1									
1	1	1	1									

(Answers removed to help you learn.)

Build Circuit

Now that you've done this, you can let LogiSim do all the hard work for you. Under "Expression" you can see the logic equations for each output, and under "Minimized" you can see the Karnaugh maps for each expression. To just have LogiSim build the circuit, click on "Build Circuit" and add it to the current project with name "control-solution".

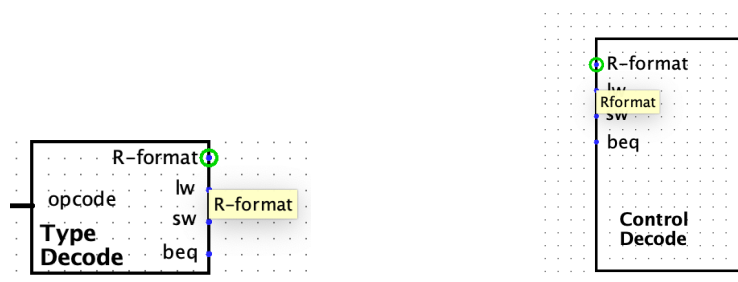
You'll now see the solution circuit in LogiSim. However, you need to copy the logic over into the real "control-decode" circuit to make it work. (Hint: select all the gates and copy them into the old circuit, but make sure the wires line up and connect to the right outputs.)

ToDo #3: Finish the Control-Decode circuit. Take a screenshot of the completed unit and add it to the report.

Testing

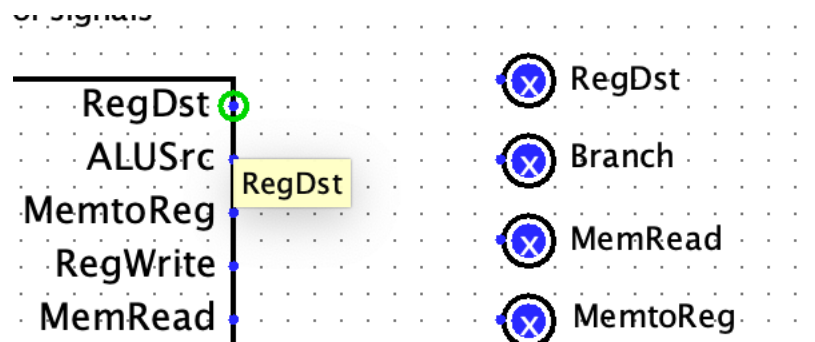
Once you have built the circuit you need to test it. Do this the same way you did for the previous circuit and make sure that the correct signals are set for each type of instruction.

Now that you've completed various units within the control unit, it's time to interconnect these units. Begin by linking the outputs of the Type Decode module to the inputs of the Control Decode module. (Hint: If you hover your pointer over the blue dots, you'll see which dots should be linked.) In the diagram below, you'll notice that the first dot on the right side of the Type Decode module aligns with the first dot on the left side of the Control Decode module. Ensure you establish a connection between these two connectors.



Next, perform a similar procedure for the outputs of the Control Decode module, connecting them to their respective destinations. For instance, referring to the diagram below, locate

the first dot on the right side of the Control Decode module, labeled as "RegDst," and establish a connection to the corresponding "RegDst" output.



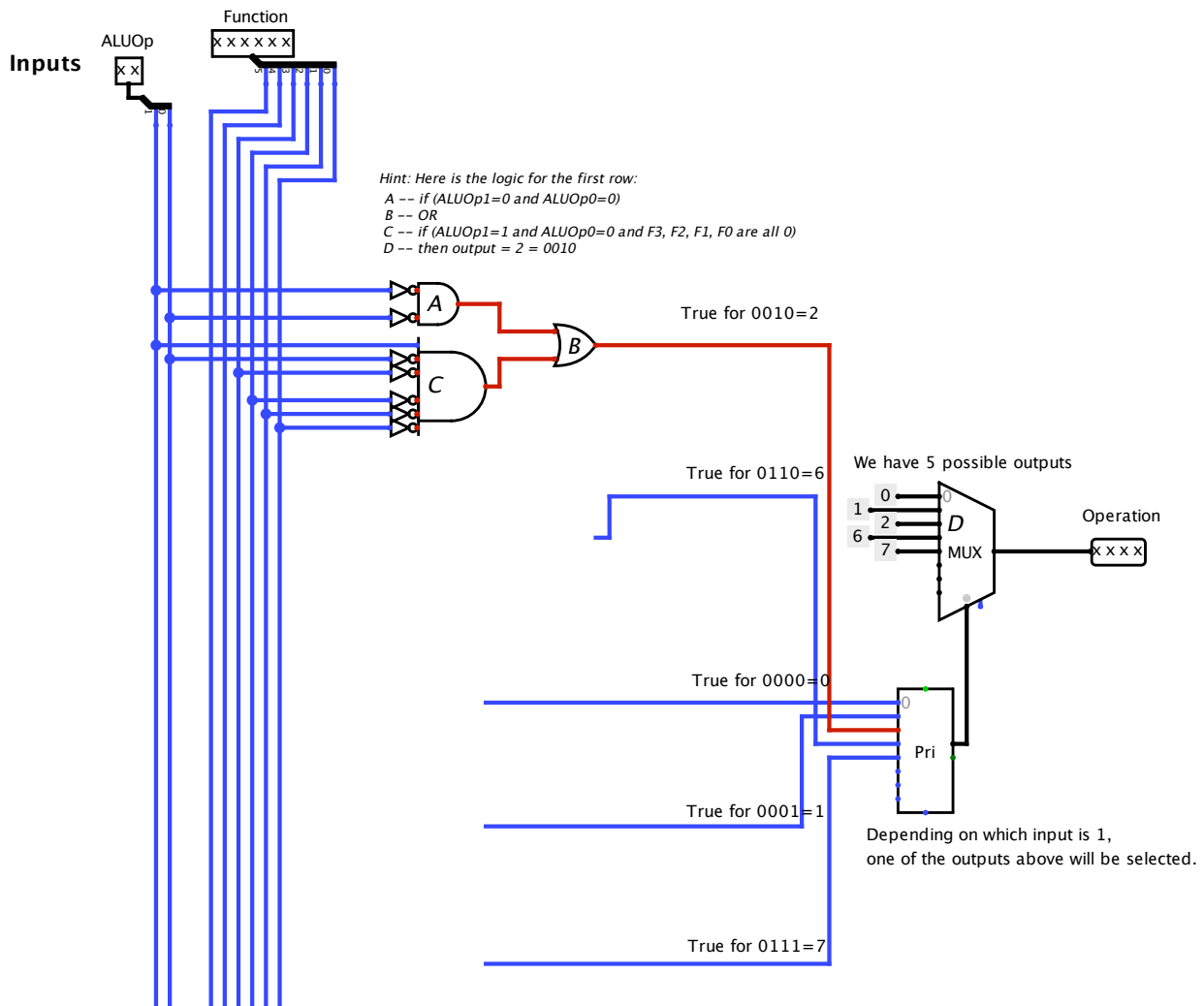
With this portion completed, it's time to test the Control Logic. Utilize the hand pointer to configure the inputs and ensure that the correct control signals are activated for each opcode

ToDo #4: Wire up your Control Logic. Take a screenshot of the completed unit and add it to the report.

Congratulations! You've now built the full control logic and tested each component. However, we strongly recommend you test the full thing. To do so, go to the control circuit and test the whole chain. E.g., put inputs into the Instruction OpCode and verify that the correct control signals are set at the output. (Trust me: time spent testing is well worth it!)

The ALU Control Unit

Take a look inside the ALU control unit, but don't be shocked. This unit has 8 inputs (2 ALUOp bits and 6 Function bits) so there are a lot of wires. In fact, it's laid out rather nicely. The inputs all go vertically down the left side and the output is on the far right. The logic that connects the inputs to the outputs goes in the middle. Spend a few minutes to figure out what the part on the right is doing to generate the output before you read the next paragraph.



(You spent a few minutes pondering the logic on the right before reading this paragraph, right?)

If you look at the logic on the right, the output is coming from a MUX (D). The MUX has constant values as inputs (0, 1, 2, 6, and 7). The MUX is driven by a priority encoder (Pri), which takes signals from the logic in the middle. What happens is that if one of the inputs to the priority encoder is true, the priority encoder will tell the MUX to select that input, which will output the constant.

To make this all work, you just need to build the logic that determines which input to the priority encoder should be true given the ALUOp and Function inputs on the left.

Also, ALU control bits based on ALUOp and Func field are provided below:

Instruction opcode	ALUOp	Instruction operation	Func field	Desired ALU action	ALU control input
LW	00	load word	XXXXXX	add	0010
SW	00	store word	XXXXXX	add	0010
Branch equal	01	branch equal	XXXXXX	subtract	0110
R-type	10	add	100000	add	0010
R-type	10	subtract	100010	subtract	0110
R-type	10	AND	100100	and	0000
R-type	10	OR	100101	or	0001
R-type	10	set on less than	101010	set on less than	0111

The truth table for controlling the ALU is also shown below:

ALUOp1	ALUOp0	F5	F4	F3	F2	F1	F0	Operation
0	0	X	X	X	X	X	X	0010
0	1	X	X	X	X	X	X	0110
1	0	X	X	0	0	0	0	0010
1	X	X	X	0	0	1	0	0110
1	0	X	X	0	1	0	0	0000
1	0	X	X	0	1	0	1	0001
1	X	X	X	1	0	1	0	0111

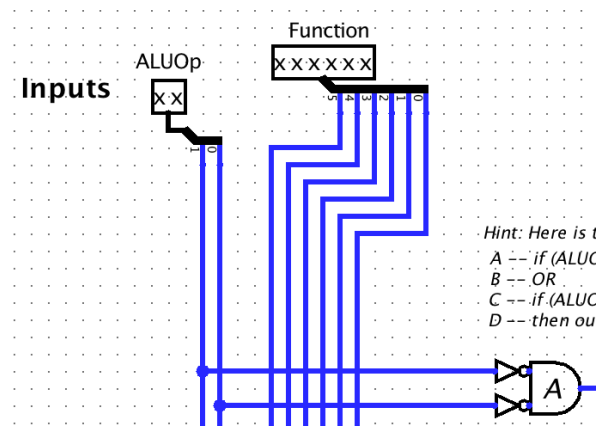
Notice the part we've already done for you in shading (the two combinations that give you a 0010 as the output). Try to write down what the logic should be for determining if the operation is 0010 before you read further. Look at the bits in the table and figure out what the conditions are.

(Really, try to do this before you read the next bit.)

Let's take a look at the first row. We have:

- ALUOp1=0, ALUOp0=0, and all Fs are X. So the logic here is:
- Operation should be 0010 whenever ALUOp1=0 AND ALUOp0=0. (We don't care about the Fs because they are Xs.)

Now look at the circuit that we gave you above:



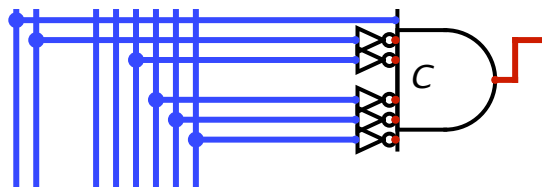
The top half (A) is an AND gate that is ANDing (NOT ALUOp1) and (NOT ALUOp0). This means the output of that AND gate will be true when ALUOp1=0 AND ALUOp0=0. (Which is, not so surprisingly, exactly what we want.)

Now what about the second shaded row where the answer is 0010? Try to work that out before going on.

The second row is:

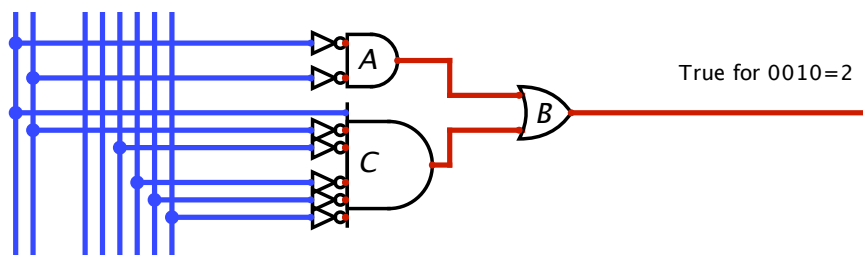
- ALUOp1=0 AND ALUOp0=0 AND F3=0 AND F2=0 AND F1=0 AND F0=0

Now look at the rest of the circuit (C) we built for you:



This is exactly that!

Now we have two cases where the output should be 0010. We need a final OR gate (B) to make sure that the output will be 0010 if either is true:



The result of the OR gate is 1 whenever the output should be 0010. To make the output be 0010 that value goes into the priority encoder, which selects the right input from the MUX to send out the results.

Note that all of the entries have Xs for both F4 and F5. That means we never use their values. So if you find yourself wiring up either F4 or F5 here you've probably made a mistake.

ToDo #5: Using the table above, complete the circuit to drive the three other possible outputs. Take a screenshot of the completed unit and add it to the report.

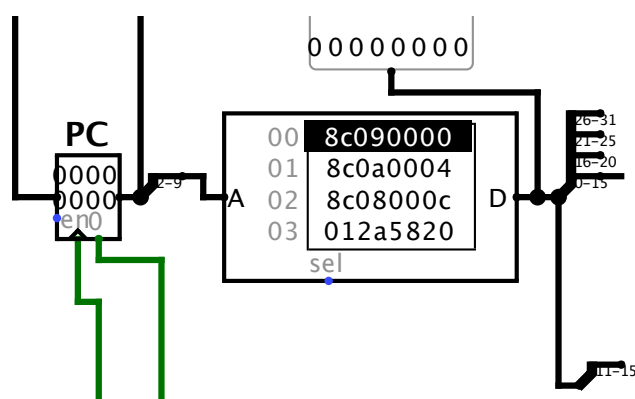
Testing

Once you have built the circuit you need to test it. Do this the same way you did for the previous circuit and make sure that the correct signals are set for each type of instruction.

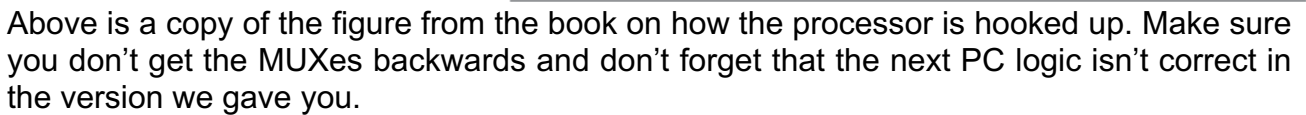
Part 2: Hooking Up the Datapath

Now that you've completed (and tested, right?) the control logic, it's time to hook up the datapath. Use the figure from the book (or datapath lectures) and your understanding of how the MIPS processor works to complete the datapath we've provided. You should not delete anything we've put in there. (There are a whole bunch of "probes" that show you the values on wires which you'll want for debugging.)

You will need to pay special attention to how the instruction is broken up for connecting the processor. Note how the output from the instruction memory is split into four parts and you can see which bits are in each part. Also note that another copy of the instruction is split up below for the immediate field. You need to make sure you know which part of the instruction goes where! You can click on the splitter to see how it is configured as well.



Selection: Splitter	
Facing	East
Fan Out	4
Bit Width In	32
Appearance	Left-handed
Bit 0	3 (Bottom)
Bit 1	3 (Bottom)
Bit 2	3 (Bottom)
Bit 3	3 (Bottom)
Bit 4	3 (Bottom)
Bit 5	3 (Bottom)
Bit 6	3 (Bottom)
Bit 7	3 (Bottom)
Bit 8	3 (Bottom)
Bit 9	3 (Bottom)
Bit 10	3 (Bottom)
Bit 11	3 (Bottom)
Bit 12	3 (Bottom)
Bit 13	3 (Bottom)
Bit 14	3 (Bottom)
Bit 15	3 (Bottom)
Bit 16	2
Bit 17	2
Bit 18	2
Bit 19	2
Bit 20	2
Bit 21	1
Bit 22	1
Bit 23	1
Bit 24	1
Bit 25	1
Bit 26	0 (Top)
Bit 27	0 (Top)
Bit 28	0 (Top)
Bit 29	0 (Top)
Bit 30	0 (Top)
Bit 31	0 (Top)



Testing it with Code

```
lw $t1, 0($zero)
lw $t2, 4($zero)
lw $t0, 12($zero)
loop:
add $t3, $t1, $t2
add $t4, $t3, $t1
sub $t0, $t0, $t2
or  $t5, $t4, $t1
slt $t6, $t4, $t5
beq $t2, $t0, loop
```

R0	\$0	Constant 0	R16	\$s0	
R1	\$at	Reserved Temp.	R17	\$s1	
R2	\$v0		R18	\$s2	
R3	\$v1	Return Values	R19	\$s3	
R4	\$a0		R20	\$s4	
R5	\$a1	Procedure arguments	R21	\$s5	
R6	\$a2		R22	\$s6	
R7	\$a3		R23	\$s7	
R8	\$t0		R24	\$t8	
R9	\$t1	Caller Save	R25	\$t9	
R10	\$t2	Temporaries: May be overwritten by called procedures	R26	\$k0	
R11	\$t3		R27	\$k1	
R12	\$t4		R28	\$gp	Reserved for Operating Sys Global Pointer
R13	\$t5		R29	\$sp	Caller Save Stack Pointer
R14	\$t6		R30	\$fp	Frame Pointer
R15	\$t7		R31	\$ra	Return Address

And a memory file that contains 1,2,3, and 4 in the first 4 words of the memory (1234.mem). To use these files you need to right-click on the instruction memory and data memories and load the appropriate files.

Now before you start running the program, figure out what that code will do.

Write down for each cycle (until the program runs out of instructions) what value is in each used register and what value and register will be written to for each instruction. (This is incredibly helpful for debugging.)

Once you've figured out what the program will do and what value will be written back to the register file after each instruction, start your program. Enable simulation, click the "Reset Program" and "Zero Register File" buttons to clear them. And then choose "Tick Once" to advance through each instruction.

When you first start simulating your first instruction will be in the processor. Make sure everything is working. Check the inputs and outputs, the MUXes, the control bits. And finally, check that the value being written back into the register file and the register being written. (These are both displayed with probes in the file so you can see them as the processor runs.)

When your program runs out of instructions you can double-check the contents of the register file by opening it up.

ToDo #7: What are the values of \$t0-\$t6 in the register file after executing the program? Take a screenshot of the register file unit (after executing the all the instructions) and add it to the report.

Debugging tips

- Don't advance past the first instruction until it is working correctly.
- Check that the values for all registers (selected register and data) are correct and that the ALU operation is correct.
- Use the finger tool to see the values on wires in the processor.

- Trace values back to the first place where they are wrong, and then go into that part to see what is going on.
- You can go inside the Control unit while your processor is simulating to see where mistakes are.
- You can view the contents of the register file by viewing inside it as well.
- Write out what you expect to happen and verify it. Don't guess.
- When you need to restart the program click Restart Program AND click Zero Reg File.
- If you reset the simulation then the memory will be zeroed and you will have to re-load the memory data.

That's it.

Turn in your completed project and the answers to the 6 Todos. The TA/Graders will test your processor file and check your questions. Don't turn in an overly messy design as the TA won't be able to understand it and will deduct points.

Grading (total 20 points)

Part 1 (10 points)

- Todo #1 (2 points): for the correct implementation of the unit and the screenshot in the report
- Todo #2 (2 point): for the completed table
- Todo #3 (2 points): for the correct implementation of the unit and the screenshot in the report
- Todo #4 (2 points): for the correct implementation of the connection and the screenshot in the report
- Todo #5 (2 points): for the correct implementation of the unit and the screenshot in the report

Part 2 (10 points)

- Todo #6 (7 points): for the correct implementation of the processor and the screenshot in the report
- Todo #7 (3 points): for the correct status of the register file after executing the program and the screenshot in the report

Revisions

Version 1.0 — Fall 2013 — David Black-Schaffer

Version 1.1 — Fall 2013 — David Black-Schaffer

Version 1.2 — Fall 2017 — Ricardo Alves, Germán Ceballos

Version 1.3 — Fall 2018 — Ricardo Alves, Germán Ceballos

Version 1.4 — Spring 2023 — Elaheh Sadredini

Version 1.5 — Summer 2025 — Elaheh Sadredini